

Same Pattern, Different Answer: A Reference Semantics and Divergence Map for GQL and SQL/PGQ Path Patterns [Experiment, Analysis & Benchmark]

Madhulatha Mandarapu
VaidhyaMegha Private Limited
Hyderabad, India
madhulatha@samyama.ai

Sandeep Kunkunuru
VaidhyaMegha Private Limited
Hyderabad, India
sandeep@samyama.ai

ABSTRACT

GQL (ISO/IEC 39075:2024) is the first international standard for a graph query language, and SQL/PGQ (ISO/IEC 9075-16:2023) embeds the same pattern-matching core, GPML, in SQL. Both fix a precise semantics for path patterns: four path modes, four selectors, quantified segments. What engines compute for those patterns has never been measured. Work on GQL is theoretical and runs no engine; cross-engine work measures performance and normalises the semantics away. We build an executable reference semantics for the GPML path core and gate it against the six worked answers printed in the standard’s own reference exposition. We then run a 17-construct suite against six releases of five products — Kùzu, DuckPGQ, Neo4j at 5.26 and 2026.04, Memgraph, Apache AGE — scoring each cell as conforming, diverging, rejected, or inexpressible in that dialect. Nine of seventeen constructs draw more than one answer across the engines that accepted them. Of 26 disagreements, 15 are silent — the query runs, returns a different multiset, raises no error — a silence ratio of 0.58. Scoring each divergence against the path mode each engine’s manual declares separates language from implementation: 9 of 15 are documented differences, 6 are departures from the standard the engine implements. A metamorphic layer needing no reference semantics finds 39 self-consistency violations, yielding two minimal defects. No engine we measured parses the order the standard’s own examples are written in, except the authors’ own, which added it after this suite reported the gap. Suite, semantics, reproducers and results are open source.

1 INTRODUCTION

For thirty years, querying property graphs meant querying a dialect. Cypher, PGQL, GSQL and Gremlin each drew a different line around what a path is and which paths a pattern returns [1]. In 2023 and 2024 that changed twice: SQL:2023 gained Part 16, SQL/PGQ [10], and GQL was ratified as ISO/IEC 39075 [11], the first international standard for a graph query language. Both build on the same pattern-matching core, GPML, whose reference exposition was written by members of the standards committee [5].

GPML makes an unusually explicit semantic commitment. A path pattern may carry a *restrictor* — TRAIL (no repeated edges), ACYCLIC (no repeated nodes), SIMPLE (no repeated nodes except that the first and last may coincide) — and a *selector* — ALL, ANY, ALL SHORTEST, ANY SHORTEST. With no restrictor the matched object is an unrestricted path, which graph theory calls a walk. Selectors are applied after restrictors. Every unbounded quantifier must sit inside the scope of one or the other, so that the answer is finite; a *bounded* quantifier needs neither.

So the standard answers, unambiguously, what $(x) - [e] -> \{1, 3\}(y)$ returns. The question this paper asks is what engines return.

Nobody has measured it. The 2023–2026 literature on GQL is uniformly theoretical: a formal semantics [8], expressive-power separations [9], and a mechanised small-step semantics with a soundness proof [19]. None of them runs a database. The one recent cross-engine paper that touches path semantics is about performance [4]: it observes in passing that Neo4j and Memgraph enforce trail during matching while Kùzu and DuckDB apply it afterwards, but all of its queries use trail semantics, so the answers are normalised and no divergence is measured. Vendor documentation offers pairwise prose lists, self-reported and not executable.

Contributions.

- (1) **An executable reference semantics** for the GPML path core (Section 3). It consults no engine, which is the point: differential testing across engines can establish that two systems disagree but not which is right. It is gated against the six worked answers the standard’s reference exposition prints (Section 3.1).
- (2) **A 17-construct conformance suite** and the first answer-level divergence map over six releases of five products (Sections 4–5).
- (3) **The silence ratio**, a statistic no prior work reports: the share of disagreements the user is not told about (Section 5).
- (4) **An attribution axis** separating a divergence that the engine’s own documentation explains from one that does not (Section 6).
- (5) **A reference-free metamorphic layer** and the two engine defects it finds, each with a standalone reproducer (Section 7).

2 BACKGROUND: THE GPML PATH CORE

A property graph is $(N, E, \rho, \lambda, \pi)$: disjoint node and edge identifier sets, a map ρ sending an edge to an ordered pair (directed) or an unordered pair (undirected), a map λ from elements to *sets* of labels, and a partial map π from an element and a property name to a value. Two distinct edges may connect the same pair of nodes, and self-loops are allowed [5].

A *path* is an alternating sequence $n_0, e_1, n_1, \dots, e_k, n_k$ that starts and ends with a node, consecutive nodes being connected by the edge between them. Traversal orientation is not part of the sequence. This is what graph theory calls a walk; the standard calls it a path, and we follow the standard.

Two consequences matter for what follows and are easy to get wrong.

A *bounded quantifier with no restrictor means walk*. The standard’s own worked example is

	Keyword	Condition
Restrictors	TRAIL	no repeated edges
	ACYCLIC	no repeated nodes
	SIMPLE	no repeated nodes, except that the first and last may be the same
	(none)	unrestricted — a walk
Selectors	ALL	every match
	ANY	one match per endpoint pair, unspecified which
	ALL SHORTEST	every match of minimum length, per endpoint pair
	ANY SHORTEST	one match of minimum length per endpoint pair

Table 1: The path restrictors and path selectors of GPML. Selectors partition the solution space on the endpoints and select within each partition; when both appear, the selector is applied after the restrictor.

```
(p WHERE p.owner='Natalia')->{1,10}
(q WHERE q.owner='Mike')->{1,10}
(r WHERE r.owner='Scott')
```

for which it states that a particular solution traverses one edge twice, and adds that this solution “fails the restrictor TRAIL (as well as SIMPLE and ACYCLIC)”. A bounded quantifier is finite on its own, so no restrictor is implied.

The quantifier binds the edge pattern, not the node pattern. In the same example the printed solution passes *through* a node that does not satisfy the segment’s node pattern on its way to one that does. Interior nodes under a quantifier are unconstrained; the node pattern binds at the end of the repetition.

3 AN EXECUTABLE REFERENCE SEMANTICS

We implement the definitions of Section 2 directly, with no optimisation and no engine in the loop. A pattern is a start node pattern followed by quantified segments; evaluation enumerates paths, filters by the restrictor, then applies the selector, in that order.

Three design points are worth stating because they are where a naive oracle goes wrong.

Unbounded quantifiers get a sound finite bound, derived per case. Under TRAIL no edge repeats, so at most $|E|$ edges; under ACYCLIC or SIMPLE at most $|N|$; under an unrestricted walk with a shortest selector, a shortest walk between two nodes is a simple path, so at most $|N| - 1$. An unbounded quantifier under an unrestricted walk with ALL is forbidden by the standard, and with ANY its admissible set is infinite, so no finite oracle exists; the implementation raises rather than guesses.

Nondeterminism is specified, not divergent. ANY and ANY SHORTEST fix how many paths are returned per endpoint partition, not which ones. The oracle therefore returns a *specification* — an exact multiset when the selector is deterministic, and otherwise an admissible set together with a per-partition count. An engine is judged against that specification. We never score a permitted choice as a divergence.

Prefix pruning must respect the restrictor’s shape. TRAIL and ACYCLIC are prefix-closed, so a prefix that already repeats an edge or a node can be pruned. SIMPLE is not: a prefix whose first and last nodes coincide is legal only if the path stops there, so pruning uses the weaker interior condition.

3.1 Gate B: reproducing the standard’s published answers

The reference exposition prints the exact path bindings six of its worked queries return over the property graph of its Figure 1. We require our implementation to reproduce all six before any engine is measured. The graph is a figure rather than text, so its Transfer sub-graph is reconstructed from the paths the paper itself prints; every edge is witnessed by at least one printed path, and the reconstruction is not asserted but tested — a wrong edge set would break at least one of the six.

The gate has teeth. An oracle that applies the segment’s node pattern at every repetition, rather than only at the end of it, fails three of the six checks — and that is a reading of Section 2 a careful person can arrive at, not a typo. An error of that shape would corrupt every cell of the map without producing a single symptom, because the map has no other yardstick to disagree with.

4 THE SUITE

Seventeen cases, each a construct where the standard makes a commitment and an implementation has freedom. The engines are Kùzu [7], DuckPGQ [18], Neo4j, Memgraph and Apache AGE. Each case carries the abstract pattern, the natural rendering in open-Cypher, and the natural rendering in SQL/PGQ. Where a dialect has no way to express the construct, the entry is empty and the cell scores INEXPRESSIBLE — absence of syntax is a fact about the language, not a defect.

Graphs are deliberately tiny, so that every expected answer is checkable by hand and every divergence is a one-screen reproducer. Six fixtures suffice: a directed two-cycle; a graph with parallel edges and a three-cycle; a self-loop with a two-cycle; a single edge; a two-hop chain whose interior node lacks the endpoints’ label; and the reconstructed Transfer sub-graph of the standard’s Figure 1.

Comparison is at the level of the multiset of endpoint identifier pairs. Every engine can produce that, which keeps the map fair across dialects that differ in how, or whether, they return path bindings. A disagreement at this level is a genuine disagreement about answers, not about presentation.

Some cases are constructed so that two different readings of the *same* query text can be told apart. On a directed two-cycle $m \rightarrow n \rightarrow m$, the pattern $\{1, 4\}$ yields four paths under WALK and two under TRAIL; on a single edge $u \rightarrow v$, an any-direction pattern

of length exactly two yields the one path u, e_1, v, e_1, u under WALK and nothing under TRAIL. openCypher has no restrictor keyword, so its rendering is the same text in both cases; the pair of cases asks which of the two meanings that one query has.

5 THE MAP

Three readings.

Nine of seventeen constructs are not portable. Counting distinct answers per construct over the engines that accepted the query, seven constructs draw exactly one answer and nine draw two or three. The three-way split is mode-walk-bounded: Kùzu returns the four walks, every Neo4j release plus Memgraph and AGE return the two trails, and DuckPGQ returns one row.

The silence ratio is 0.58. Of 26 disagreements, 15 are silent. A rejection is a portability problem the user sees at once; a divergence is a portability problem that ships. Neo4j 5.26 and Memgraph reject nothing in the suite, so every one of their disagreements is silent — $S_2 = 1.00$. This is the number we think matters most and it is not reported anywhere else. It is also the most stable thing in the table: it sits between 0.58 and 0.61 under all three aggregations, and the numerator — fifteen silent divergences — is identical in every one.

Three products reproduce the standard’s own worked example. On the TRAIL query whose three bindings the reference exposition prints, both Neo4j releases, Memgraph and Apache AGE return exactly three. That is a useful positive control on the whole apparatus, independent of our oracle.

Three engines take the standard’s keywords, and no two spell them the same way. The suite asks each engine at startup which spelling of the restrictor and selector prefixes it parses, rather than assuming (Section 4). Kùzu, DuckPGQ, Memgraph and Apache AGE take neither. The three that take anything disagree with each other:

The last column is the sharp one. ISO/IEC 39075 heads a path pattern with the restrictor and binds the variable to the whole thing — MATCH TRAIL $p = (\dots)$ — and the worked examples in the reference exposition are written that way. **No external engine we measured parses it.** The single engine that does is the authors’ own, and it does so because this suite reported the gap and its maintainers, who are the authors, fixed it (Section 8). That is worth exactly one conclusion and no more: the order is implementable, so its absence elsewhere is a choice rather than an obstacle.

The one version pair we have suggests slow convergence, not none. Neo4j 5.26.30 and 2026.04.0 agree on 14 of 17 constructs. The three that move all go the same way: ACYCLIC becomes expressible and correct on two cells, and SIMPLE is attempted and rejected. Nothing regressed, and nothing else changed. One vendor pair is not a trend, but it is the only version-over-version evidence in this paper and it points at adoption proceeding construct by construct rather than in one release.

6 ATTRIBUTION: LANGUAGE DIFFERENCE OR IMPLEMENTATION DEFECT?

A divergence from the ISO reference is not by itself a defect. openCypher specifies that a variable-length relationship pattern uses relationship uniqueness — no relationship traversed twice within

one MATCH — which is TRAIL. An openCypher engine that returns the trails is doing exactly what its language says. So we score every answer a second time, against the path mode the engine’s own manual declares: TRAIL for Neo4j, Memgraph and Apache AGE; WALK for Kùzu, which documents its deviation from openCypher explicitly; and, for DuckPGQ, nothing, because the dialect it implements is the standard.

Nine of the fifteen divergences are documented language differences. Every Kùzu and every openCypher divergence falls in this class: those engines do precisely what they say they do, and the gap is between the dialect and the standard, not between the manual and the binary. The remaining six are DuckPGQ’s, and DuckPGQ has no second reading to appeal to.

This split is the practical payload. For a user porting a query, a documented language difference is a fact to design around; a departure from the implemented standard is a bug to report. Aggregating the two — as a bare conformance scoreboard would — tells neither audience what to do.

7 A LAYER THAT NEEDS NO REFERENCE

The map is only as persuasive as our reading of the standard, and where a dialect deliberately deviates it says nothing about whether the engine is *correct*. So we add a second, weaker but unarguable layer: metamorphic relations [3, 17] that hold under *every* path mode in the standard, because no restrictor mentions the quantifier bounds and every path has exactly one length.

For $A(lo, hi)$ the answer multiset of a quantified pattern:

$$\begin{aligned} M1 \quad & A(lo, hi) \subseteq A(lo, hi + 1) \\ M2 \quad & A(lo, hi) \subseteq A(lo - 1, hi) \\ M3 \quad & A(lo, hi) = \biguplus_{k=lo}^{hi} A(k, k) \end{aligned}$$

An engine that breaks one of these contradicts itself, whatever it believes a path is.

Thirty-nine violations, all in two engines. They reduce to two defects.

Kùzu 0.11.3: a lower bound of zero double-counts returns to the start. On a graph with a self-loop at x and a two-cycle $x \rightarrow y \rightarrow x$, the engine reports $*0..0 = \{x\}$, $*1..1 = \{x, y\}$ and $*2..2 = \{x, x, y\}$, but $*0..2 = \{x \times 6, y \times 2\}$ where the sum of the three is $\{x \times 4, y \times 2\}$. Every path whose end node is the start node is returned twice, and the zero-length path is dropped, whenever the lower bound is zero.

DuckPGQ: the quantified pattern is not the standard’s, in three ways. On a directed two-cycle $m \rightarrow n \rightarrow m$: $\{0, 0\}$ returns a one-hop answer rather than the zero-length path; $\{2, 2\}$ returns nothing although m, e_1, n, e_2, m is a length-two path — and the engine’s own $\{0, 2\}$ does find m at distance two; and a quantified pattern collapses the multiset of paths to one row per reachable endpoint, while the unquantified single-edge pattern keeps the multiset. The second of these is self-contradictory and needs no appeal to the standard at all.

Both are reproduced by standalone scripts in the artifact, each under sixty lines with no dependency on the rest of the suite.

	kuzu	duckpgq	neo4j	neo4j-2026	memgraph	apache-age	samyama-graph
mode-walk-bounded	✓	✗	✗	✗	✗	✗	✗
mode-trail-bounded	✗	!	✓	✓	✓	✓	✓
mode-acyclic-bounded	-	!	-	✓ †	-	-	✓ †
mode-simple-bounded	-	!	-	! †	-	-	✓ †
walk-revisits-same-edge	✓	✗	✗	✗	✗	✗	✗
quant-zero-lower-bound	✓	✗	✓	✓	✓	✓	✓
quant-zero-zero	✓	✗	✓	✓	✓	✓	✓
parallel-edges	✓	✓	✓	✓	✓	✓	✓
self-loop-trail	✗	!	✓	✓	✓	✓	✓
self-loop-acyclic	-	!	-	✓ †	-	-	✓ †
direction-any	✓	✓	✓	✓	✓	✓	✓
direction-left	✓	✓	✓	✓	✓	✓	✓
interior-node-unconstrained	!	!	✓	✓	✓	!	✓
selector-all-shortest	✓	!	✓	✓	✓	✓	✓
selector-any-shortest	-	✗	✓ †	✓ †	-	-	✓ †
selector-any	-	✗	✓ †	✓ †	-	-	✓ †
paper-trail-unbounded	✗	!	✓	✓	✓	✓	✓

✓ conforms ✗ diverges ! rejects - inexpressible † answered only via the GQL prefix keywords

Figure 1: The divergence map. Rows are constructs, columns engines. Each cell also carries a glyph, so the figure survives greyscale printing and colour-vision deficiency. Every cell is an exact multiset comparison, run three times to confirm the engine is deterministic; no engine was nondeterministic. The last column is the authors’ own engine, shown for completeness and excluded from every statistic in Table 2 and Section 6.

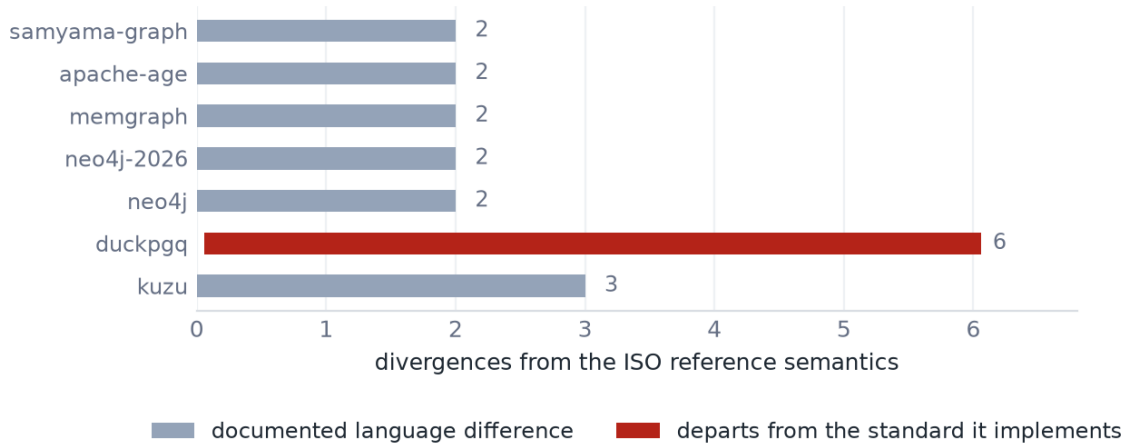


Figure 2: Divergences from the ISO reference, split by whether the engine’s own documentation accounts for them.

8 LIMITATIONS AND HONEST NEGATIVES

The suite is narrow by design. Seventeen constructs over the path core. Writes, transactions, schema, aggregation, subqueries and

engine	conforms	diverges	rejects	inexpressible	S_1	S_2
Kùzu 0.11.3	8	3	1	5	0.27	0.75
DuckPGQ (DuckDB 1.4.1)	3	6	8	0	0.67	0.43
Neo4j 5.26.30	12	2	0	3	0.14	1.00
Neo4j 2026.04.0	14	2	1	0	0.12	0.67
Memgraph 5.9.0	10	2	0	5	0.17	1.00
Apache AGE 1.8.0	9	2	1	5	0.18	0.67
headline*	44	15	11	15	0.25	0.58
<i>Samyama-Graph v1.8.0[‡] (ours)</i>	15	2	0	0	0.12	1.00

Table 2: S_1 is divergences over the cells the engine answered. S_2 , the *silence ratio*, is divergences over divergences plus rejections: the share of disagreements the user is not told about. The authors’ own engine is listed below the rule and is excluded from the totals and from every other statistic in this paper. * the headline aggregate takes one row per product, at its newest measured version, so that measuring Neo4j twice does not weight that vendor twice; the 5.26 row is shown but not counted. Over all six external rows the figures are $S_1 = 0.23$, $S_2 = 0.61$, and over the five products with Neo4j at 5.26 they are $S_1 = 0.26$, $S_2 = 0.60$. The count of silent divergences is 15 under every one of the three; only the denominator moves. [‡] the authors’ own engine, measured at its published release like every other row, and excluded from the totals. Its release carries fixes this suite prompted, which is why it scores as it does; see Section 8 before reading anything into that row. Five of its 15 conforming cells are answered only through the standard’s prefix keywords, which four of the six external releases do not parse at all; for Neo4j 2026.04 that count is 4 of 14.

	restrictor	selector	RESTRICTOR p = . . .
Neo4j 5.26.30	neither spelling	both spellings	—
Neo4j 2026.04.0	quantified only	both spellings	rejected
<i>Samyama-Graph v1.8.0 (ours)</i>	<i>legacy only</i>	<i>legacy only</i>	accepted

Table 3: Which spelling of the standard’s path prefixes each engine parses, for the engines that parse any. “Legacy” is $-[:E*1..3]->$; “quantified” is $(()-[:E]->())\{1,3\}$. “—”: that release takes no restrictor, so there is nothing to place before the variable. Neo4j 2026.04 refuses the restrictors in the legacy form and says so, naming the legacy quantifier in the error. Three engines, three different subsets, and no overlap in the last column.

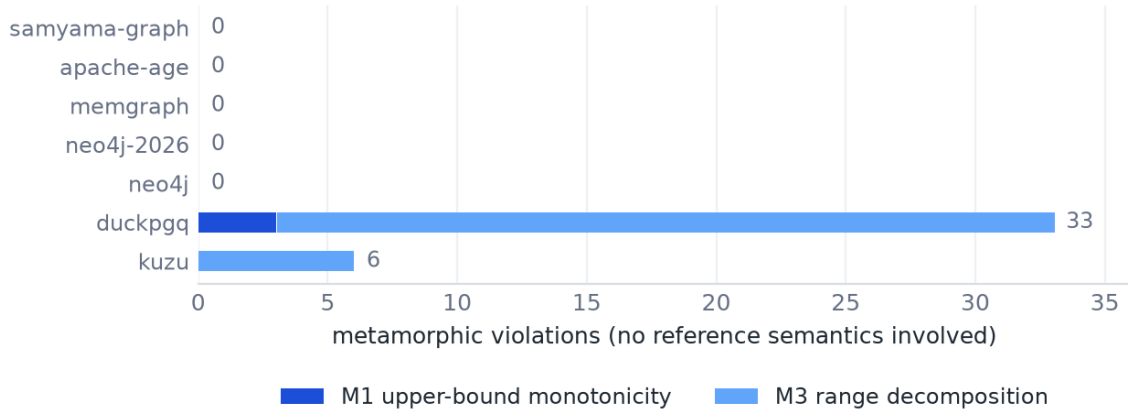


Figure 3: Metamorphic violations over six graphs and all quantifier pairs with $hi \leq 4$. Both Neo4j releases, Memgraph, Apache AGE and the authors’ own release are clean; the authors’ row enters no count. No query errored on any engine in this run, so these counts are complete rather than lower bounds.

graph construction are out of scope, as is performance: this artifact contains no timing number and makes no claim about speed.

Two engines are not GQL engines and are not scored as failing one. Neo4j, Memgraph, Apache AGE and Kùzu implement openCypher

and its descendants, not GQL. The map places them against the ISO semantics because that is the only common yardstick, and Section 6 exists precisely so that this placement is not mistaken for a conformance verdict. Neo4j is measured at two releases, 5.26.30 and 2026.04.0, so that dialect-version drift is visible rather than folded into a single product row (Section 5).

Comparison is at endpoint level. We compare multisets of endpoint pairs, not of full path bindings, so that dialects which cannot return path objects are still measurable. This can only *under-count* divergence: two engines agreeing on endpoints might still disagree on which paths produced them. Every divergence we report is therefore real; some divergences are certainly missed.

A query that errors cannot violate a relation. The metamorphic layer skips a relation when any query it needs fails rather than counting the failure, so a crash can hide a violation but never add one. The artifact records every such refusal, and in this run **no query errored on any engine**: the counts above are complete, not lower bounds. On a run where that is not true the artifact says so per engine.

Our reconstruction of the standard’s Figure 1 could in principle be wrong. It is validated by six independent published answers, which is strong but not a proof. The reconstruction and its witnesses are published with the artifact so the check can be repeated.

We did not find a divergence everywhere we looked. Seven constructs draw a single answer across every engine that accepted them, including parallel edges, both direction forms, and ALL SHORTEST. Those are portable today, and saying so is part of the result.

Conflict of interest, and how to read our row. This work was produced at Samyama, which develops one of the engines measured. It is measured the way every other engine is: at its published release, v1.8.0. Its row sits below the rule in Table 2 and enters no total, no aggregate, and no figure count. No engine’s output, ours included, was used to derive any expected answer; every expected answer comes from the reference semantics of Section 3, which never consults a database.

That release carries fixes this suite prompted. An earlier release scored 8 of 17 conforming with 22 metamorphic violations. The suite’s findings went to that engine’s maintainers — who are the authors — as issues with reproducers, and the fixes shipped: a quantifier with lower bound zero that collapsed the multiset of paths to one row per reachable end node, the path restrictors and selectors, the restrictor-then-variable order, and unbounded memory growth on a graph of two nodes and three edges, which had made an ordinary read query a denial of service.¹

So the row is not evidence that this engine is better. No other engine’s maintainers saw these results before their measured release shipped; ours did, and acted on them within days. What the row is evidence of is narrower and, we think, more useful: every defect this suite reported was a real defect, and each was fixable in an ordinary engineering cycle rather than requiring a redesign. The same is available to every vendor here — the suite, the reproducers and the issues are public.

The one axis on which our release is not flattered is the one this paper argues matters most. Its silence ratio is 1.00: it rejects nothing in the suite, so both of its divergences are silent, and a user meets them as wrong answers rather than as errors. Neo4j 2026.04 refuses a construct out loud where we simply answer differently, and on that axis it is ahead of us.

9 RELATED WORK

GQL and SQL/PGQ theory. Deutsch et al. [5] give the reference exposition of GPML; Francis et al. [8] the formal semantics of GQL; Gheerbrant et al. [9] expressive-power results for both languages; and Thimmaiah et al. [19] a mechanised small-step semantics with a soundness proof for a read-only fragment. All four verify against the specification. None runs an engine, which is the gap this paper fills. Thimmaiah et al. [19] in particular is complementary: a mechanised semantics is a stronger artifact than ours, and connecting it to the suite is the natural next step.

Path semantics. Restricting paths to simple paths or trails to keep answers finite goes back to Mendelzon and Wood [13]; the complexity landscape for trail queries is mapped by Martens et al. [12]. That literature explains why the standard offers restrictors at all; it does not say what implementations do.

DBMS testing. The oracle problem for database engines is attacked by pivoted query synthesis, non-optimising reference engine construction and query partitioning [14–16], and guided by plan coverage [2]. Those oracles are *internal*: they compare a system to itself under a transformation, which finds bugs but cannot adjudicate between two systems that disagree. Our reference semantics is external and adjudicates; our metamorphic layer is internal in exactly the classical sense and is included because it survives disagreement about the standard.

Cross-engine measurement. Correa and Riedewald [4] compare engines on path queries for performance, normalising to trail semantics. Erling et al. [6] and the LDBC suites measure throughput on fixed workloads. Neither measures whether the answers agree.

10 CONCLUSION

Two international standards now define what a graph path pattern means, and the definitions are precise. We made those definitions executable, gated the result against the standard’s own published answers, and ran it against six releases of five products.

The same pattern draws more than one answer for nine of seventeen constructs, and more than half of the disagreements are silent. Splitting divergence by whether the engine’s own documentation explains it turns a scoreboard into two actionable lists: a portability surface for users, and six concrete defects for one engine’s maintainers. A reference-free metamorphic layer adds two more defects, one of them a case where an engine’s answer to $\{2, 2\}$ contradicts its own answer to $\{\emptyset, 2\}$.

The suite is small enough to read in an afternoon and is open source. We would like it to become the thing an engine runs before it claims to implement GQL.

Artifact. Reference semantics, suite, adapters, reproducers, raw results and figure code are at <https://github.com/samyama-ai/gpml>.

¹[https://github.com/samyama-ai/samyama-graph/issues/1140, 1141, 1148, 1183](https://github.com/samyama-ai/samyama-graph/issues/1140,1141,1148,1183).

conformance, under Apache-2.0. `./run.sh` starts the engines in Docker and regenerates every number and figure in this paper.

REFERENCES

- [1] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan L. Reutter, and Domagoj Vrgoč. 2017. Foundations of Modern Query Languages for Graph Databases. *Comput. Surveys* 50, 5 (2017).
- [2] Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *45th IEEE/ACM International Conference on Software Engineering (ICSE)*.
- [3] T. Y. Chen, S. C. Cheung, and S. M. Yiu. 1998. Metamorphic Testing: A New Approach for Generating Next Test Cases. *Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology* (1998).
- [4] Diego Rivera Correa and Mirek Riedewald. 2026. Efficient Path Query Processing in Relational Database Systems. [arXiv:2604.02553](#).
- [5] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Fred Zemke. 2022. Graph Pattern Matching in GQL and SQL/PGQ. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. 2246–2258. [arXiv:2112.06217](#).
- [6] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat-Pérez, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC Social Network Benchmark: Interactive Workload. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*.
- [7] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoglu. 2023. Kuzu Graph Database Management System. In *13th Conference on Innovative Data Systems Research (CIDR)*.
- [8] Nadime Francis, Amelie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoč. 2023. A Researcher’s Digest of GQL. In *26th International Conference on Database Theory (ICDT)*.
- [9] Amelie Gheerbrant, Leonid Libkin, Liat Peterfreund, and Alexandra Rogova. 2025. GQL and SQL/PGQ: Theoretical Models and Expressive Power. *Proceedings of the VLDB Endowment* (2025). [arXiv:2409.01102](#).
- [10] ISO/IEC. 2023. ISO/IEC 9075-16:2023 — Information technology — Database languages SQL — Part 16: Property Graph Queries (SQL/PGQ). International Standard.
- [11] ISO/IEC. 2024. ISO/IEC 39075:2024 — Information technology — Database languages — GQL. International Standard.
- [12] Wim Martens, Matthias Niewerth, and Tina Trautner. 2020. A Trichotomy for Regular Trail Queries. In *37th International Symposium on Theoretical Aspects of Computer Science (STACS)*.
- [13] Alberto O. Mendelzon and Peter T. Wood. 1995. Finding Regular Simple Paths in Graph Databases. *SIAM J. Comput.* 24, 6 (1995), 1235–1258.
- [14] Manuel Rigger and Zhendong Su. 2020. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.
- [15] Manuel Rigger and Zhendong Su. 2020. Finding Bugs in Database Systems via Query Partitioning. In *Proceedings of the ACM on Programming Languages (OOPSLA)*.
- [16] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [17] Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering* 42, 9 (2016), 805–824.
- [18] Daniel ten Wolde, Tavneet Singh, Gabor Szarnyas, and Peter Boncz. 2023. DuckPGQ: Efficient Property Graph Queries in an Analytical RDBMS. In *13th Conference on Innovative Data Systems Research (CIDR)*.
- [19] Aditya Thimmaiah, Tong-Nong Lin, and Milos Gligoric. 2026. MGQL: An Executable, Small-Step Semantics of GQL. [arXiv:2608.24565](#).